

CS2109S: Introduction to AI and Machine Learning

Lecture 4: Intro to Machine Learning & Decision Trees

3 September 2026

[PolLEV.com/rizki](https://polly.com/rizki)

Capstone Project will be released this week!



Objective: Apply the AI & ML knowledge that you have/will learn throughout the course.

- Develop an agent to play Grid Adventure V1.
 - Fully-observable, deterministic, single-agent.
 - Image observation.
- Can use any techniques: search and/or ML.
- Grading: performance on graded test cases.
- Due date: **end of semester (week 13)**.
- Released early to let you work on it gradually.

You are here



1	2	3	4	5	6	7	8	9	10	11	12	13
"Classical" AI				"Classical" ML					"Modern" ML			

Search Algorithms

- Uninformed search: BFS, DFS
- Informed search: A*
- Local Search: Hill Climbing
- Adversarial search: Minimax

"Classical" ML

- Decision Trees
- Linear/Logistic Regression
- Regularization and Kernels
- Neural networks

"Modern" ML

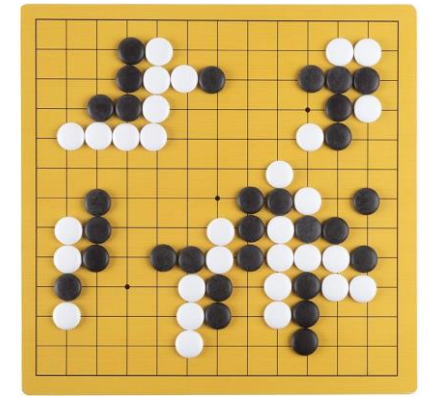
- Deep Learning
- Sequential Data
- Intro to unsupervised (deep) learning

Outline

- **Machine Learning**
 - Problems
 - Supervised Learning
- Decision Trees
 - Data
 - Model
 - Learning
 - Pruning
 - Real-World Data

Problems and a new paradigm

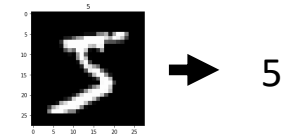
- Some problems are inherently difficult to solve, e.g., they have astronomically large state space. Examples:
 - The game of Go
 - Protein folding.
- Other problems are hard to solve because they're difficult to express to a computer in the way it can understand. Ex:
 - Recognizing numbers in an image
 - Answering open-ended questions
 - Tasks requiring complex reasoning.
- **A new paradigm:** Build an agent that **learns** a function to identify patterns in data and use them to make decisions or solve problems.



Credit: Amazon.sg

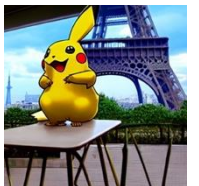
$b \approx 250, m \approx 150$ for “reasonable” games

Time complexity: $b^m = 250^{150} \approx 10^{360}$



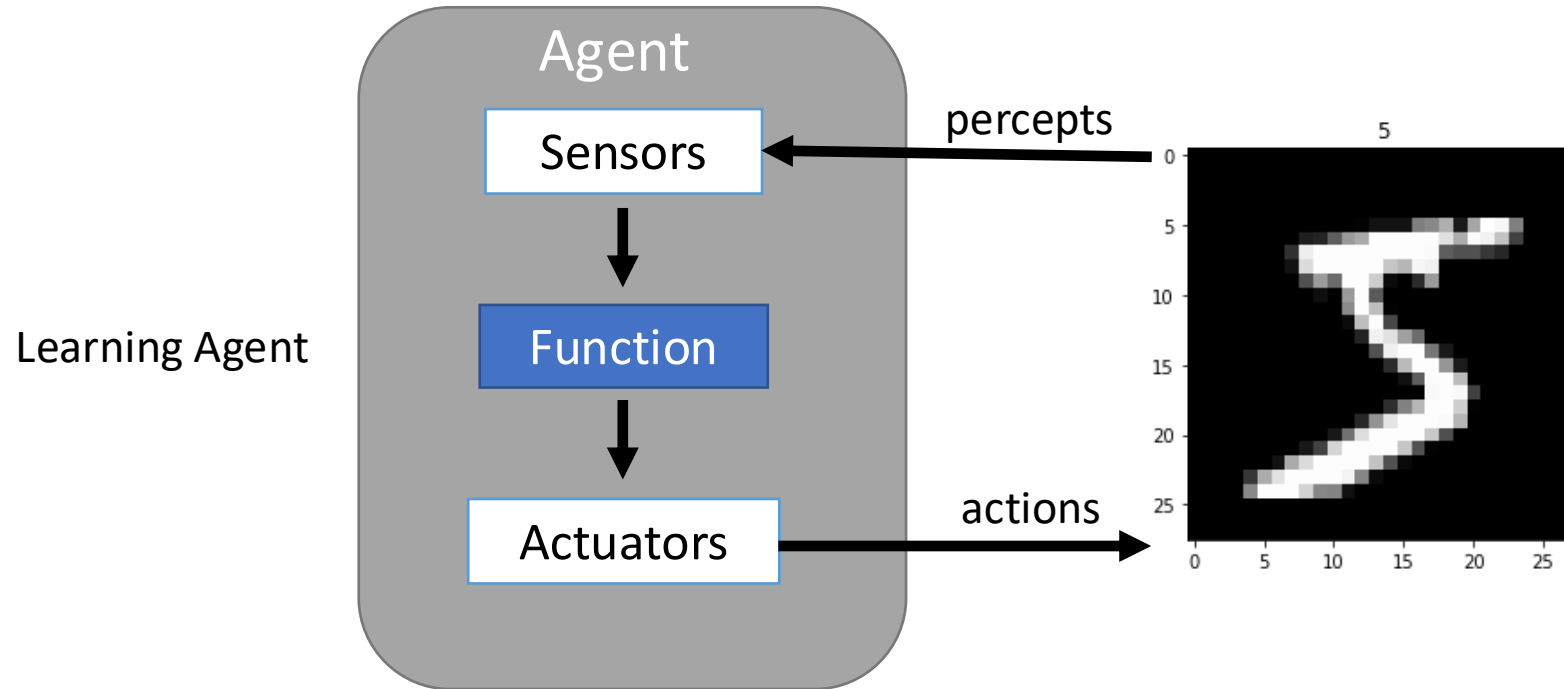
“What is AI?” → “AI is ...”

“Pikachu in the Eiffel tower” →



Designing an Agent

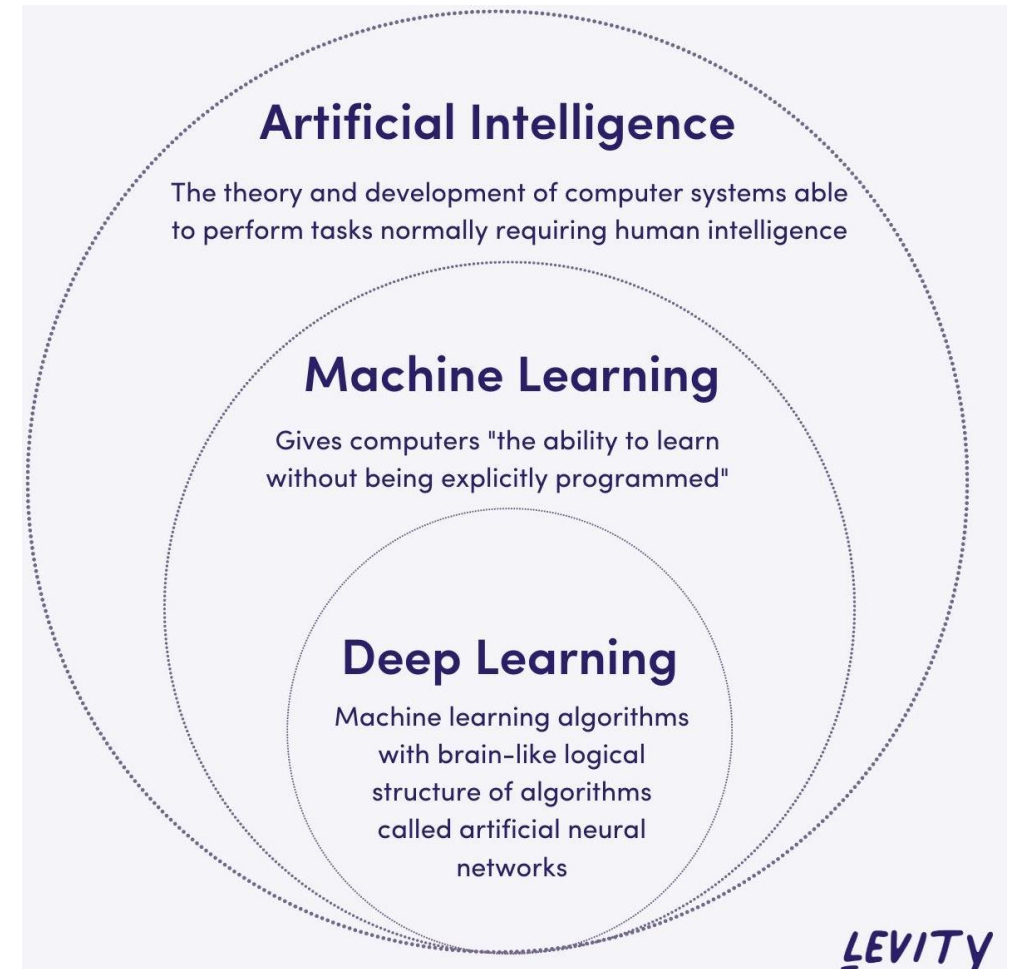
for problems where: - The function is difficult to specify, or
- Solutions are difficult to compute (in general)



Machine Learning

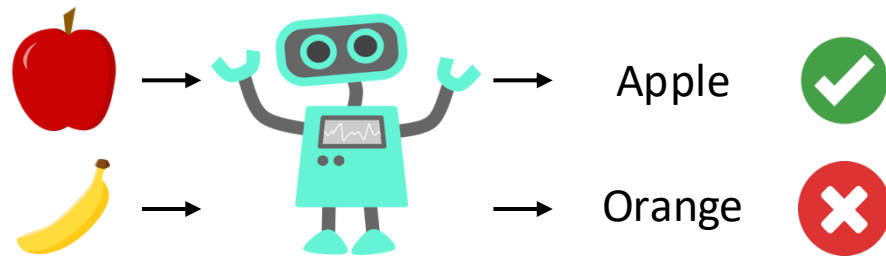
Machine learning (ML) is a subfield of AI that gives computers **the ability to learn and improves performance from data/experience/observations.**

It involves developing algorithms that can **learn** from and **make predictions or decisions** based on **data**.

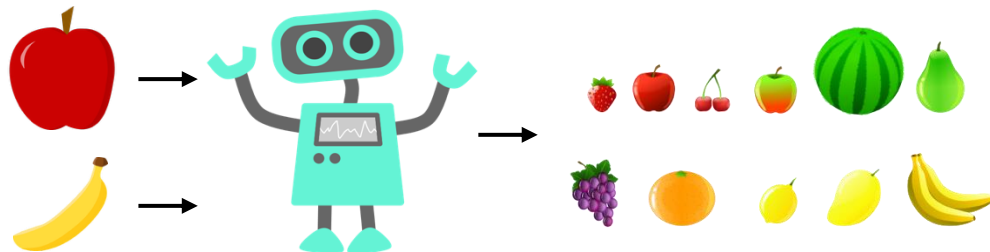


Credit: levity.ai

Machine Learning – Types



Supervised
"Teacher's feedback"



Unsupervised
"No feedback"

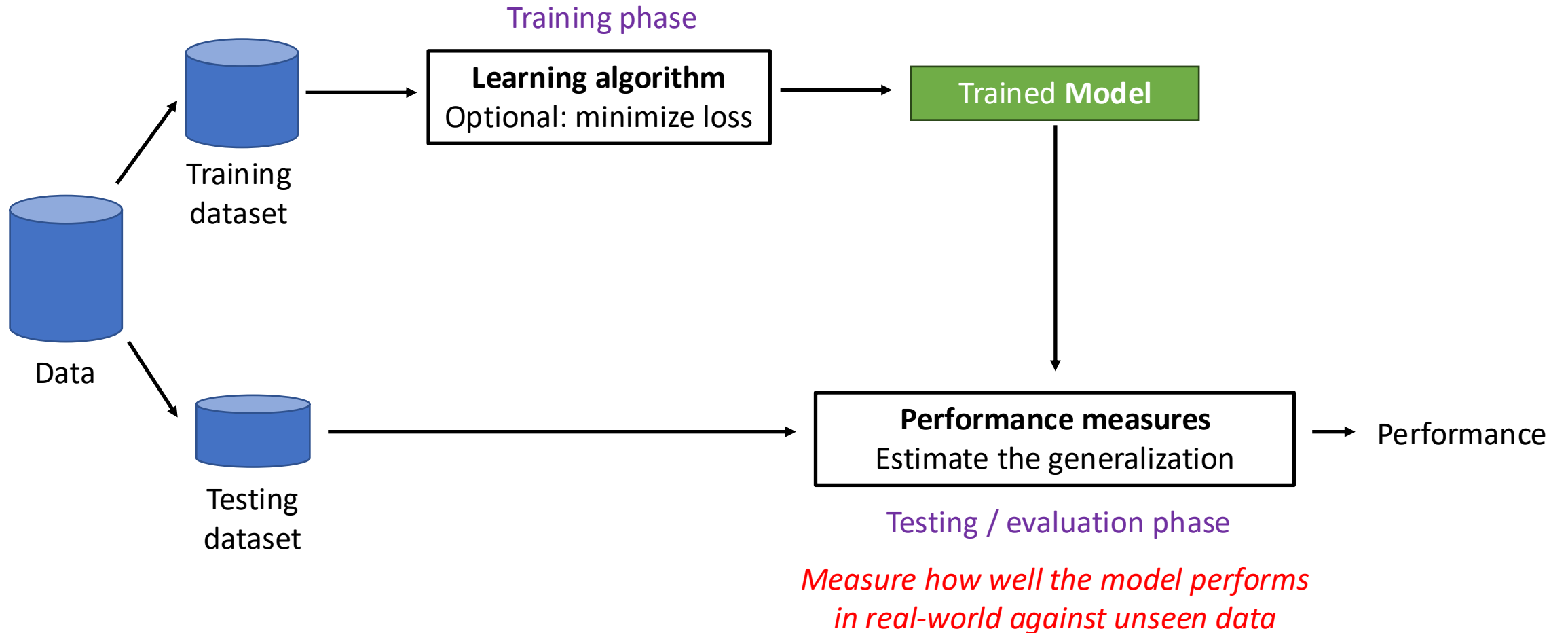
- **Supervised Learning:** Learn from labeled data (input-output pairs) to learn a mapping from inputs to outputs.
 - Example: classify whether an image contains a picture of a banana or apple
- **Unsupervised Learning:** Learn from unlabeled data (input only) to find patterns or structure.
 - Example: group images of fruits into 2, 3, 4, ... groups, based on features like color, shape, etc
- **Semi-supervised Learning**
- **Reinforcement Learning**

Supervised Learning

Problem-solving using supervised learning involves the following components:

- **Data:** A set of **input-output** pairs that the model learns from.
 - We assume that the data are drawn *independently* and *identically distributed* (i.i.d.) from a *fixed* underlying **probability distribution**.
 - *Independence*: each sample is drawn without dependence on the others.
 - *Identical distribution*: all samples are drawn from the same probability distribution.
- **Model:** A function that predicts the output based on the input features.
- **Loss:** A function that is often used by a **learning algorithm** to assess the model's ability in making predictions over a set of data.
 - Loss is often used to guide learning.

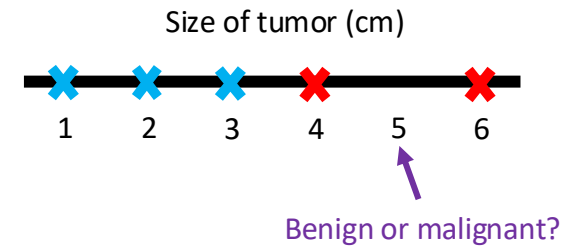
Supervised Learning



Tasks

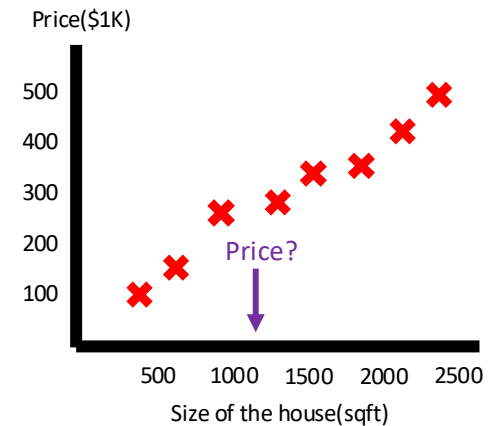
- **Classification:**

- Objective: predict a discrete label/class/category from a set of predefined classes/categories based on input features.
- The output variable (target) is a **categorical value**.
- Example: cancer prediction, spam detection, document categorization



- **Regression:**

- Objective: predict a continuous numerical value based on input features.
- The output variable (target) is a **real number**.
- Example: house price prediction, temperature prediction, sales forecasting



Data

Dataset in supervised learning is represented as a set of pairs $(x^{(i)}, y^{(i)})$, where

- **$x^{(i)}$ is the input (features)** vector of d length/features of the i -th data point.
 - $x^{(i)}$ features may be real number, categorical values, etc
 - In general, $x^{(i)}$ can be multi-dimensional array of features.
- **$y^{(i)}$ is the label (target)** for the i -th data point.
 - $y^{(i)} \in \mathbb{R}$ for regression
 - $y^{(i)} \in \{1, 2, \dots, K\}$ for classification with K classes

Dataset D with n examples can be represented as

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\}$$

Model

- A **model** or **hypothesis** refers to a function that maps from inputs to outputs $h: X \rightarrow Y$ and that can be learned by a **learning algorithm**.
- We are interested in finding a model $h(x)$ that best predicts the outputs of given inputs in a dataset.
- Example (coming up later in this course):
 - Decision trees, linear/logistic models, neural networks

Performance Measure & Loss Functions

Performance measure and **loss functions** are functions that take in a dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$ and a model h and return a number representing how well h predicts the mapping $x^{(i)} \rightarrow y^{(i)}$.

They may share the same functional form!

- **Performance measure** is what you use to judge the model at evaluation/testing on *test data*.
- Technically: **loss function** is a function that quantifies the difference between the model's predictions $\hat{y}^{(i)}$ and the true outputs $y^{(i)}$.

It is computed on *training data* and used to guide learning algorithm.

Performance Measure – Regression

Given a dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$ where $y^{(i)} \in \mathbb{R}$, common methods of evaluating the performance of a model h are:

Mean **squared** error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

Mean **absolute** error (MAE):

$$MAE = \frac{1}{n} \sum_{i=1}^n \|\hat{y}^{(i)} - y^{(i)}\|$$

Here, $h(x^{(i)}) = \hat{y}^{(i)}$.

Note: MSE and MAE can also be used as loss functions (as we will see later).

Performance Measure – Classification

Given a dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$ where $y^{(i)} \in \{1, 2, \dots, K\}$, common methods of evaluating the performance of a model h are:

Accuracy (average correctness):

$$Accuracy = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{\hat{y}^{(i)} = y^{(i)}}$$

Where $h(x^{(i)}) = \hat{y}^{(i)}$.

Confusion matrix, precision, recall, F1-score (next slide).

Performance Measure – Classification

Task: predict cancer malignant (Positive) or benign (Negative).

No	Predicted $\hat{y}$	Actual y	
1	Positive	Positive	TP
2	Positive	Positive	
3	Negative	Positive	FN
4	Negative	Positive	
5	Negative	Positive	
6	Negative	Negative	TN
7	Negative	Negative	
8	Negative	Negative	
9	Negative	Negative	FP
10	Positive	Negative	

Confusion Matrix

		True Label	
		Positive	Negative
Predicted Label	Positive	2 True Positive (TP)	1 False Positive (FP)
	Negative	3 False Negative (FN)	4 True Negative (TN)

$$\text{Accuracy} = \frac{TP+TN}{TP+FN+FP+TN}$$

Precision

$$P = TP / (TP+FP)$$

If false positives are costly
e.g., spam prediction (removed emails)

Recall

$$R = TP / (TP+FN)$$

If false negatives are costly
e.g., cancer prediction (missed cancer)

F1 Score

(Harmonic mean between precision and recall)

$$F1 = \frac{2}{\frac{1}{P} + \frac{1}{R}}$$

Balance precision & recall

[True/False] [Positive/Negative]

“The prediction is [Positive/Negative] and it is [True/False]”

Note: Precision/recall/F1 require a "positive" class, so for $K > 2$ they must be computed per class and then averaged.

Learning Algorithms

A **learning algorithm** A learns from training data D_{train} , consisting of pairs $(x^{(i)}, y^{(i)})$, to find a model h that predicts $y^{(i)}$ given $x^{(i)}$ well, i.e., $h(x^{(i)}) \approx y^{(i)}$.

- Most algorithms use a loss function to guide this process.
- The same algorithm can usually be applied to different data, models, and loss functions.
- Example (later in this course): decision tree learning, gradient descent

Supervised Learning Diagram

Training

Training Set $\{(x^{(1)}, y^{(1)}), \dots, (x^{(n_{train})}, y^{(n_{train})})\}$



Learning Algorithm A $\leftarrow$ Loss Function
Find $h(x)$ that best fits the training set



Model $h(x)$



Testing

Test Set
 $\{(x^{(1)}, y^{(1)}), \dots, (x^{(n_{test})}, y^{(n_{test})})\}$

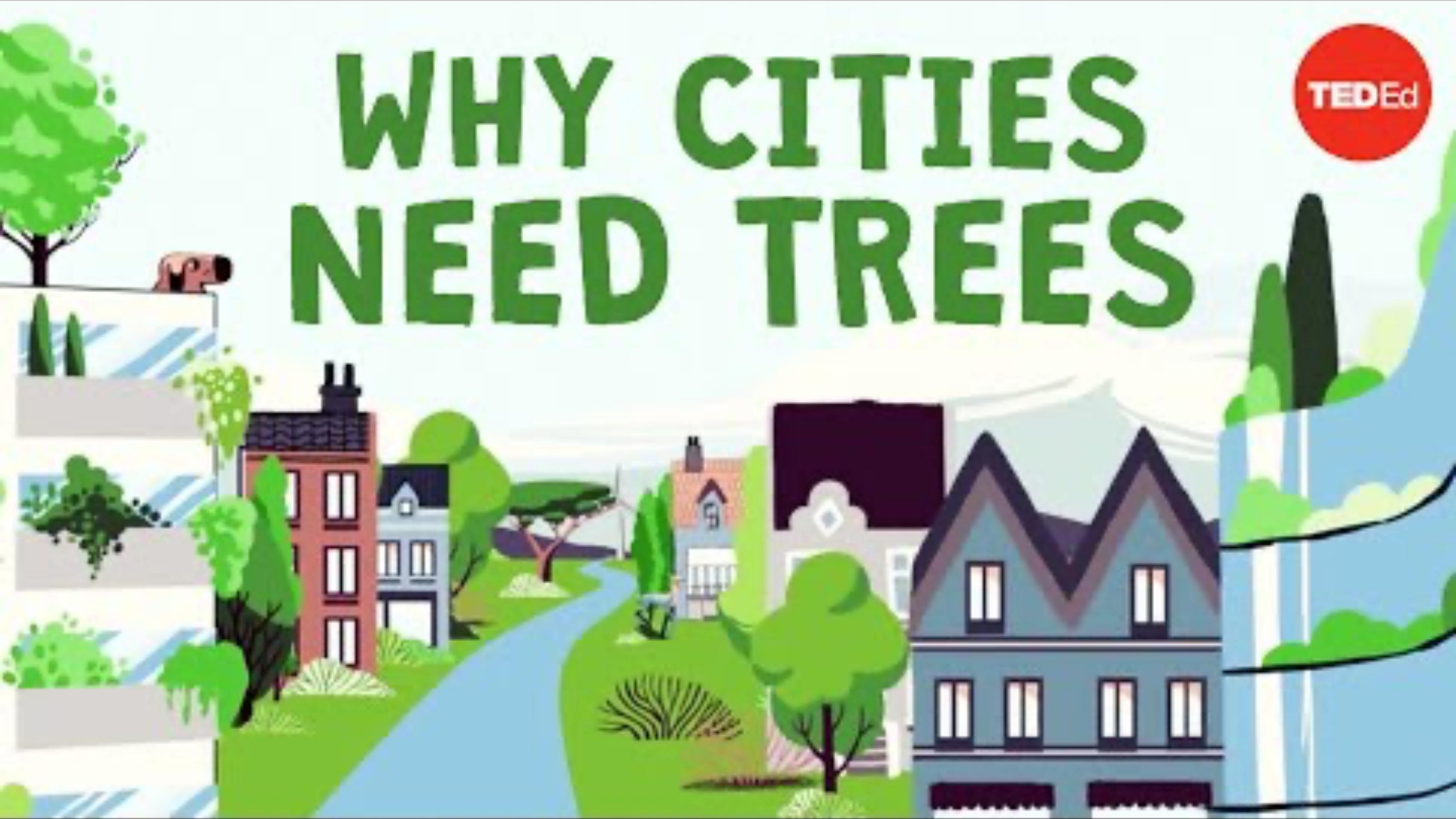


Performance Measure
Check $h(x)$ fits in test set
(Generalization)

Break

WHY CITIES NEED TREES

TEDEd



Outline

- Machine Learning
 - Problems
 - Supervised Learning
- **Decision Trees**
 - Data
 - Model
 - Learning
 - Pruning
 - Real-World Data

Example: Hiring Decision

Given a candidate for a job. Would you hire them?

Hire?

?

For the hiring decision, it makes sense to look at **attributes of the candidate**.

Experience	Interview	Skills	Hire?
Junior	Good	High	?

What is the appropriate hiring decision, given the data of previous hires?

It also makes sense to look at **data of previous hires**.

No	Experience	Interview	Skills	Hire?
1	Senior	Good	High	Yes
2	Advanced	Bad	High	Yes
3	Junior	Good	Mid	Yes
4	Junior	Bad	High	No
5	Junior	Bad	Mid	No

Data

Suppose:

- We are given n data points.
- Each data point consists of **features** and a **target** variable.
- The features are described by a feature vector or row with d number of **categorical** features (attributes).
- The target is $\{1, 2, \dots, K\}$, where K is the number of classes.

Data – Math

Suppose:

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\},$$

where for all $i \in \{1, \dots, n\}$.

- Features: $x^{(i)} \in \{c_1, c_2, \dots, c_m\}^d$ for some categories $c_1, c_2, \dots, c_m$.
- Targets: $y^{(i)} \in \{1, 2, \dots, K\}$ for K classes.

Task

Suppose we are given another data point $x \in \{c_1, c_2, \dots, c_m\}^d$ and no target. Based on the dataset, find a function that predicts the target $y \in \{1, 2, \dots, K\}$ for that x .

This is a classification task.

Example: Hiring Decision (Formalized)

Given a candidate x with features experience (x_1), interview result (x_2), and skills (x_3),
What is the output (y)?

	Experience (x_1)	Interview (x_2)	Skills (x_3)	Hire? (y)
$x^{(1)}$	Senior	Good	High	Yes
$x^{(2)}$	Advanced	Bad	High	Yes
$x^{(3)}$	Junior	Good	Mid	Yes
$x^{(4)}$	Junior	Bad	High	No
$x^{(5)}$	Junior	Bad	Mid	No

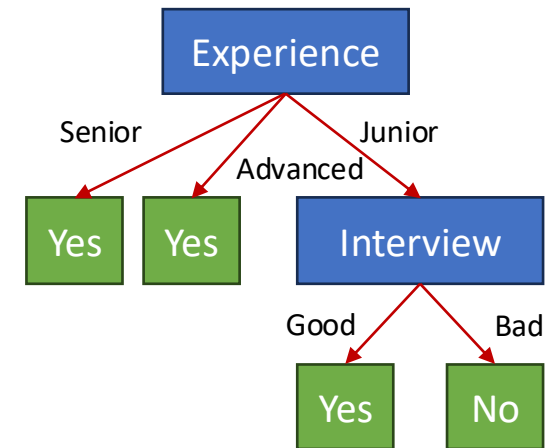
Decision Tree Model

A **decision tree** is a model that implements a function with:

- Inputs: a vector of **categorical** values $x \in \{c_1, c_2, \dots, c_m\}^d$
- Outputs: a decision or prediction $y \in \{1, 2, \dots, K\}$.

A decision tree is a tree structure where:

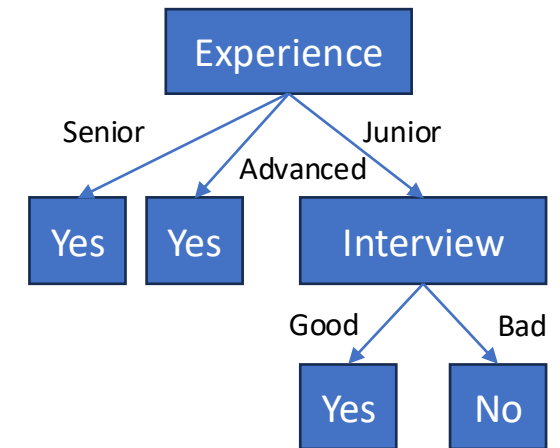
- **Internal nodes** represent test attributes/features
- **Branches** represent the outcomes of those tests
- **Leaf nodes** give predictions



Decision Tree Model – Prediction

Starting from the root, sequentially check each node's attribute and follow the matching branch until you reach a leaf. Essentially, it's a **nested if-else**.

```
if Experience == "Senior": return "Yes"  
if Experience == "Advanced": return "Yes"  
if Experience == "Junior":  
    if Interview == "Good": return "Yes"  
    if Interview == "Bad": return "No"
```



Q: Given a candidate with **Junior** experience, **Good** interview, and **High** skills, would you hire them?

Optimal Decision Tree Model

There generally exist multiple **distinct decision tree models** that are all consistent with the training data—all classifying every training example correctly.

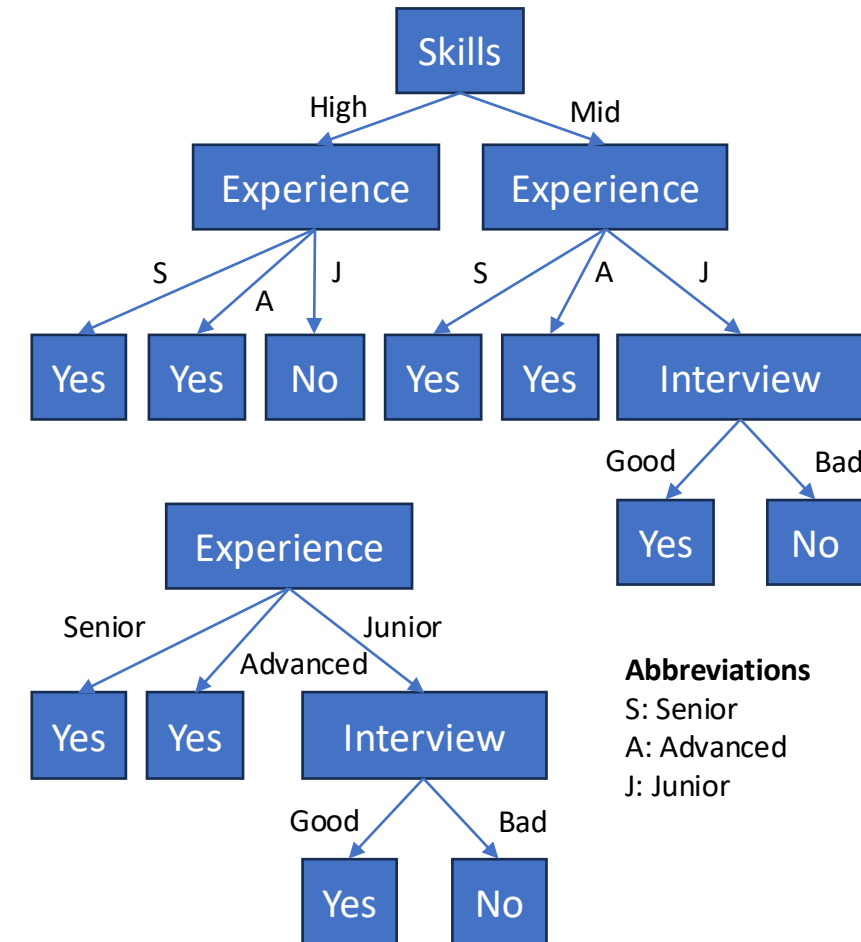
Prefer a "compact" decision tree: the decision tree has the smallest possible number of nodes.

Also called the **optimal decision tree**.

Why?

- It's hard for a small tree to fit the data
- Therefore, one that fits probably captures the genuine pattern! It's not due to chance.

No	Experience	Interview	Skills	Hire?
1	Senior	Good	High	Yes
2	Advanced	Bad	High	Yes
3	Junior	Good	Mid	Yes
4	Junior	Bad	High	No
5	Junior	Bad	Mid	No

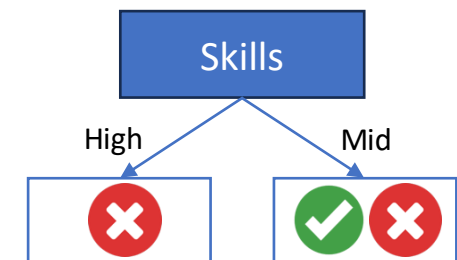
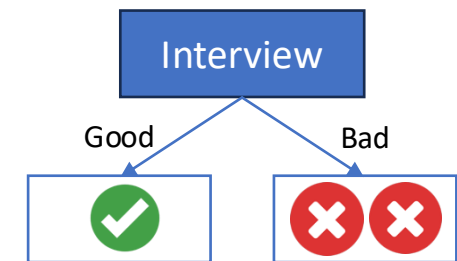


Learning a Decision Tree Model

Theorem: Finding an **optimal decision tree** consistent with all training examples is **NP-Hard**.

Solution: Use greedy heuristics to make it tractable—at each step, choose the feature with the highest **information gain** to split the data as **decisively** as possible.

No	Interview	Skills	Hire?
3	Good	Mid	Yes
4	Bad	High	No
5	Bad	Mid	No



Background: Entropy

Entropy quantifies the uncertainty or randomness of a random variable.

Let Y be a random variable with outcomes values $y_1, \dots, y_k$ and probabilities $P(y_1), \dots, P(y_k)$. Then, the entropy is defined as:

$$H(Y) = H(P(y_1), \dots, P(y_k)) = - \sum_{i=1}^k P(y_i) \log_2 P(y_i)$$

If the outcomes are decisions, entropy measures the level of **uncertainty** (or **indecisiveness**) about a decision.

Background: Entropy for Binary Outcomes

For a dataset which contains p positive and n negative outcomes.

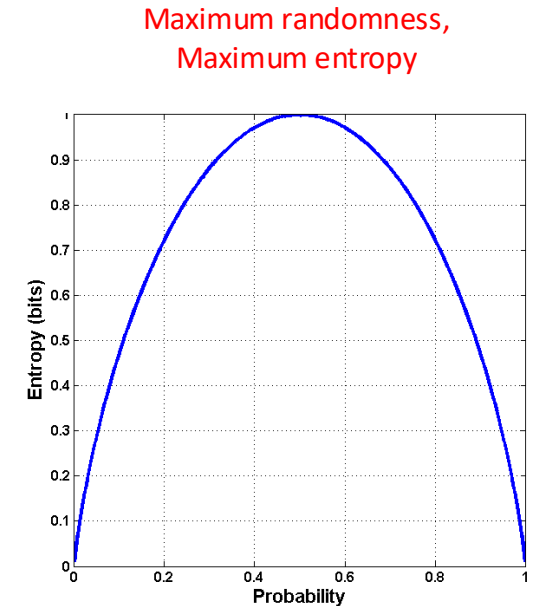
Probabilities:

$$P(\text{pos}) = \frac{p}{p+n} \text{ and } P(\text{neg}) = \frac{n}{p+n}$$

Entropy:

$$H(Y) = H(P(\text{pos}), P(\text{neg})) = - \sum_{y_i \in \{\text{pos}, \text{neg}\}} P(y_i) \log_2 P(y_i)$$

$$= - \left(\frac{p}{p+n} \log_2 \frac{p}{p+n} \right) - \left(\frac{n}{p+n} \log_2 \frac{n}{p+n} \right)$$



Background: Entropy – Example

Hire? (y)
Yes
Yes
Yes
No
No

Probabilities:

$$P(pos) = \frac{p}{p+n} = \frac{3}{5}$$

$$P(neg) = \frac{n}{p+n} = \frac{2}{5}$$

Entropy:

$$H(Y) = H(P(pos), P(neg))$$

$$= H\left(\frac{3}{5}, \frac{2}{5}\right)$$

$$= -\left(\frac{3}{5} \log_2 \frac{3}{5}\right) - \left(\frac{2}{5} \log_2 \frac{2}{5}\right)$$

$$= 0.971$$

Background: Specific Conditional Entropy

Specific conditional entropy $H(Y|X = x)$ is the entropy of Y when X is fixed at a particular value x . For a variable Y with k possible outcomes, it is defined as:

$$H(Y|X = x) = H(P(y_1|x), \dots, P(y_k|x))$$

Experience	Hire? (y)
Senior	Yes
Advanced	Yes
Junior	Yes
Junior	No
Junior	No

For example, the specific conditional entropy for hiring decision given that “Experience” is taking a value “Junior” is:

$$\begin{aligned}
 H(Y|Experience = \text{Junior}) &= H\left(\frac{1}{3}, \frac{2}{3}\right) \\
 &= -\left(\frac{1}{3} \log_2 \frac{1}{3}\right) - \left(\frac{2}{3} \log_2 \frac{2}{3}\right) \\
 &= 0.918
 \end{aligned}$$

Background: Conditional Entropy

Conditional entropy $H(Y|X)$ measures how much uncertainty about Y remains once we know X . It's defined as:

$$H(Y|X) = \sum_{x \in X} P(X = x) H(Y|X = x)$$

When outcomes are decisions, it captures how **undecided** we remain about a decision after another variable is known.

Background: Conditional Entropy – Example

Experience	Hire? (y)
Senior	Yes
Advanced	Yes
Junior	Yes
Junior	No
Junior	No

Abbreviations

S: Senior

A: Advanced

J: Junior

$$\begin{aligned}
 H(Y|Experience) &= \sum_{x \in \{S, A, J\}} P(Experience = x) H(Y|Experience = x) \\
 &= \frac{1}{5} \times 0 + \frac{1}{5} \times 0 + \frac{3}{5} \times 0.918 = 0.551
 \end{aligned}$$

$$P(Experience = S) = \frac{1}{5}$$

$$P(Experience = A) = \frac{1}{5}$$

$$P(Experience = J) = \frac{3}{5}$$

$$H(Y|Experience = S) = H(1) = 0$$

$$H(Y|Experience = A) = H(1) = 0$$

$$H(Y|Experience = J) = H\left(\frac{1}{3}, \frac{2}{3}\right) = -\left(\frac{1}{3} \log_2 \frac{1}{3}\right) - \left(\frac{2}{3} \log_2 \frac{2}{3}\right) = 0.918$$

Information Gain

Information gain measures how much our **uncertainty** about a decision drops once we know the value of a specific variable.

The formula is:

$$IG(Y; X) = \underbrace{H(Y)}_{\text{Uncertainty before}} - \underbrace{H(Y|X)}_{\text{Uncertainty after selecting a variable}}$$

Example (using data from the previous slide):

$$IG(Y; Experience) = H(Y) - H(Y|Experience) = 0.971 - 0.551 = 0.42$$

Decision Tree Learning – Overview

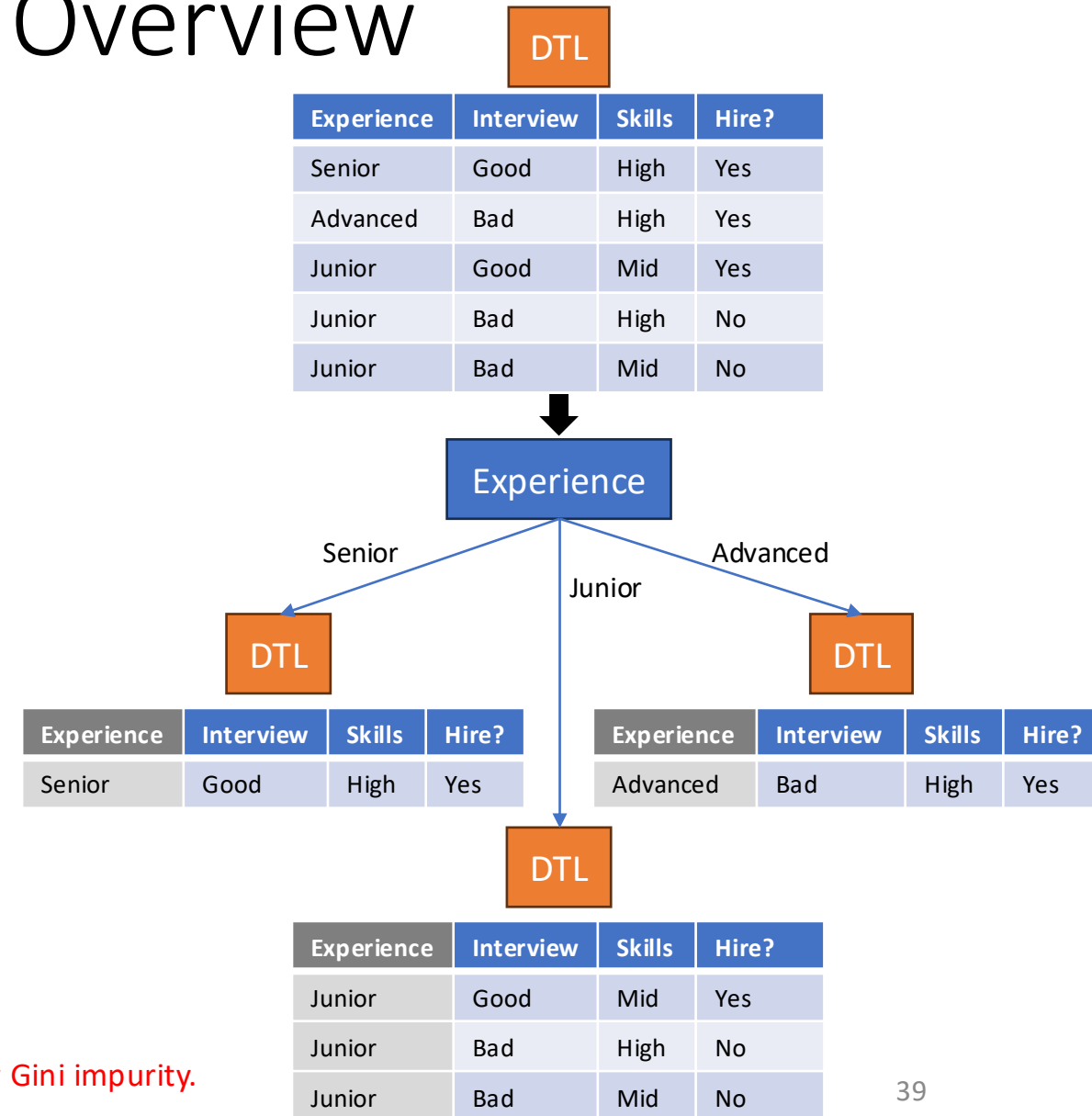
Top-down, greedy, recursive algorithm.

- **Top-down:** Build the decision tree from the root.
- **Greedy:** At each step, choose the local best attribute to split (e.g., by immediate information gain), without considering future splits.
- **Recursive:** For each value of the chosen attribute, apply DTL again to the corresponding subset of data.

Stopping criteria:

- No more data or attributes, or
- All remaining data predicts the same class.

Note: there are other methods of choosing the best attribute to split, e.g., by Gini impurity.



Decision Tree Learning – Functions

- **choose_attribute(attributes, rows)**
 - Accepts a set of **attributes** (table columns) in the data and the table **rows** containing the data points.
 - Compute the **information gain** of each attribute in **attributes**.
 - Returns the attribute that has the highest information gain.
- **majority_decision(rows)**
 - Accepts the table **rows** containing the data points.
 - Returns the most common decision in the data points.

Decision Tree Learning – Base Cases

```
def DTL(rows, attributes, default):  
    if rows is empty: return default  
    if rows have the same classification:  
        return classification  
    if attributes is empty:  
        return majority_decision(rows)  
    ...
```

1

Experience	Interview	Skills	Hire?
------------	-----------	--------	-------

 No data available → Default

2

Interview	Hire?
Bad	No
Bad	No

 Same decision for all rows → “No”

3

Hire?
Yes
Yes
Yes
No
No

 Ran out of attributes
3 “Yes”, 2 “No” → “Yes”

When do the base cases happen?

- 1 No training data for a particular attribute value. DTL called with no data.
- 2 Most common case. After splitting data according to values.
- 3 Inconsistent data. Noisy data. Insufficient attributes to describe data.

Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    > best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$

Decision entropy:

$$H(Y) = H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$

DTL

Conditional entropy:

$$H(Y|Experience) = \frac{1}{5}H(1) + \frac{1}{5}H(1) + \frac{3}{5}H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.551$$

$$H(Y|Interview) = \frac{2}{5}H(1) + \frac{3}{5}H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.551$$

$$H(Y|Skills) = \frac{3}{5}H\left(\frac{1}{3}, \frac{2}{3}\right) + \frac{2}{5}H\left(\frac{1}{2}, \frac{1}{2}\right) = 0.951$$

Information gain:

Tie is broken arbitrarily

$$IG(Y; Experience) = H(Y) - H(Y|Experience) = 0.42$$



$$IG(Y; Interview) = H(Y) - H(Y|Interview) = 0.42$$

$$IG(Y; Skills) = H(Y) - H(Y|Skills) = 0.02$$

Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute

    1 for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        2 subtreei = DTL(subrowsi,
                           attributes - best_attribute,
                           majority_decision(subrowsi))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

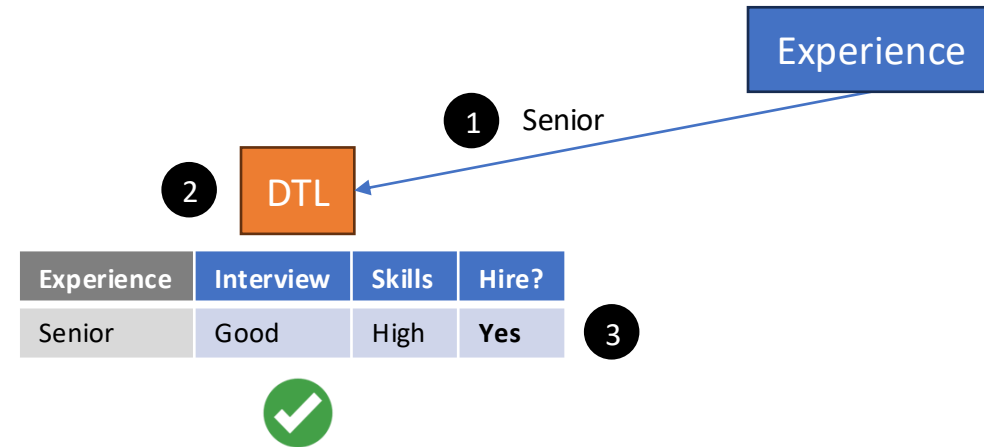
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
    > add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

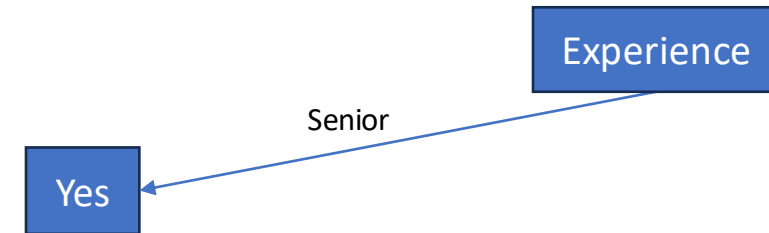
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

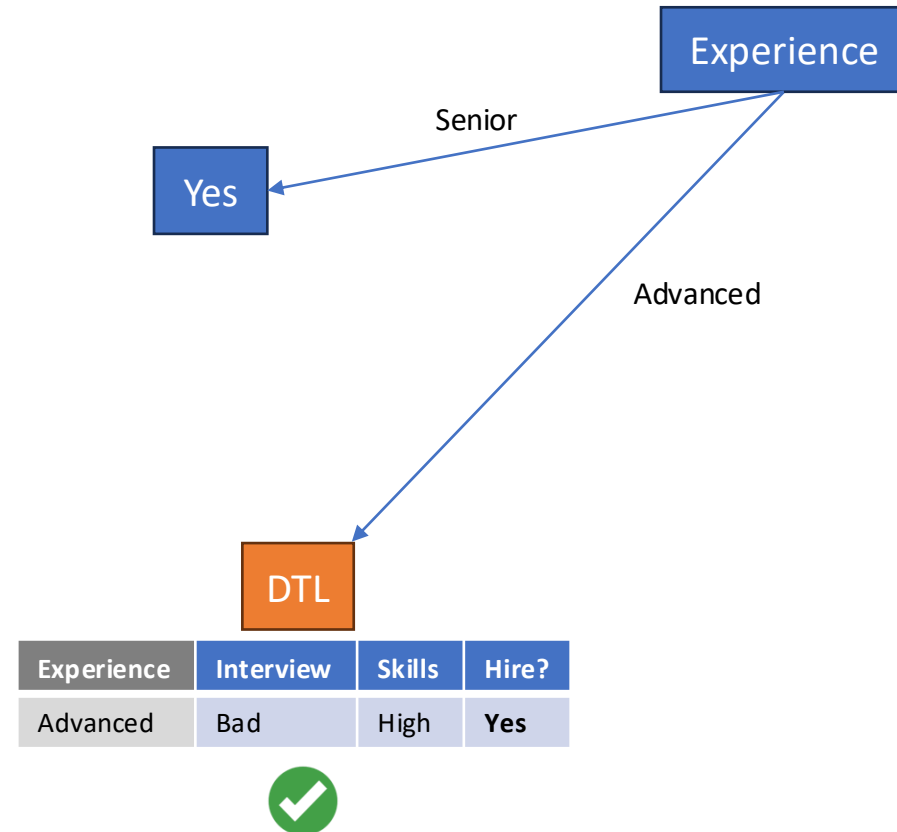
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):  
    if rows is empty: return default  
    if rows have the same classification:  
        return classification  
    if attributes is empty:  
        return majority_decision(rows)  
  
    best_attribute = choose_attribute(attributes, rows)  
    tree = a new decision tree with root best_attribute  
    for valuei of best_attribute:  
        subrowsi = {rows with best_attribute = valuei}  
        subtreei = DTL(subrowsi,  
                        attributes - best_attribute,  
                        majority_decision(rows))  
        add subtreei as a branch to tree with label valuei  
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

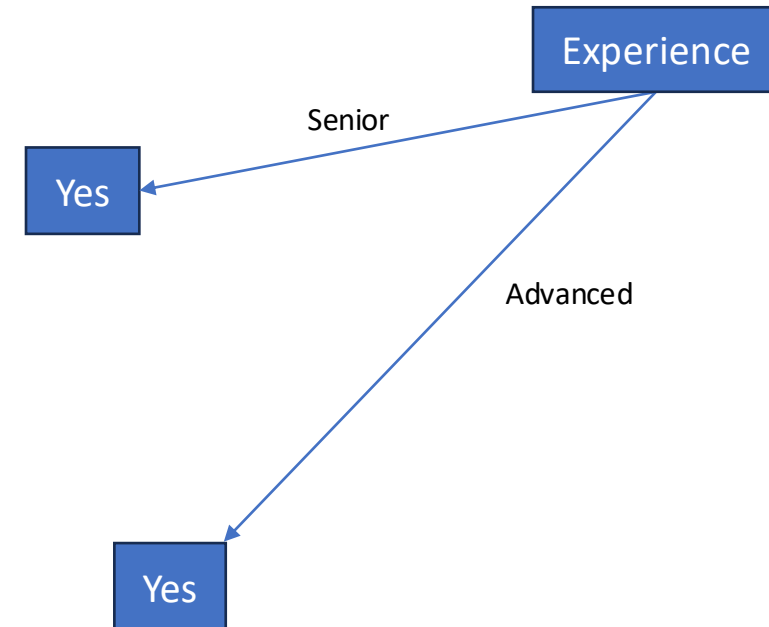
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

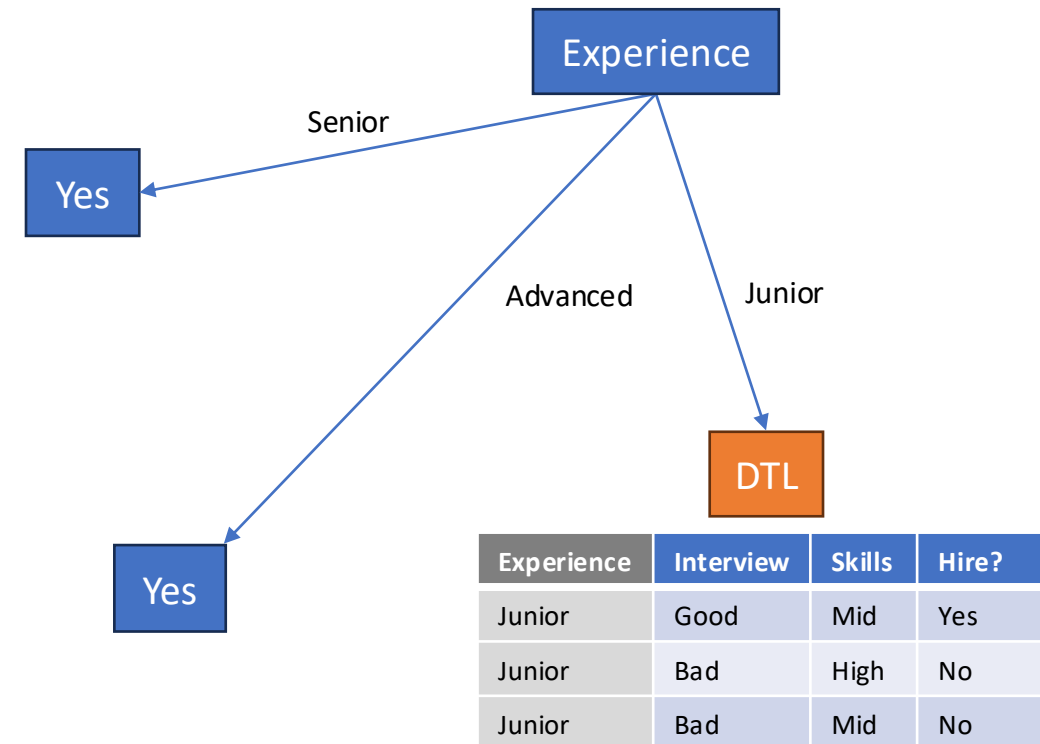
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

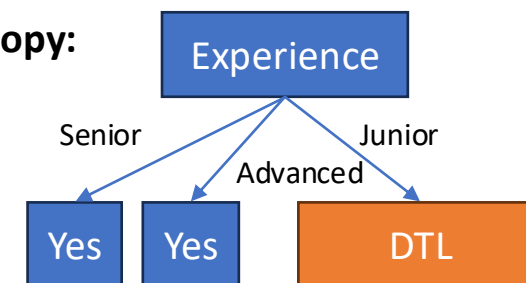
$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$

Decision (specific conditional) entropy:

$$H(Y|Experience = J)$$

$$= H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$



Conditional entropy:

$$H(Y|Experience = J, Interview)$$

$$= \frac{1}{3}H(1) + \frac{2}{3}H(1) = 0$$

$$H(Y|Experience = J, Skills)$$

$$= \frac{2}{3}H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{1}{3}H(1) = 0.667$$

Experience	Interview	Skills	Hire?
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Abbreviation

J: Junior

Information gain:

$$IG(Y|Experience = J; Interview) = 0.918$$

$$IG(Y|Experience = J; Skills) = 0.252$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

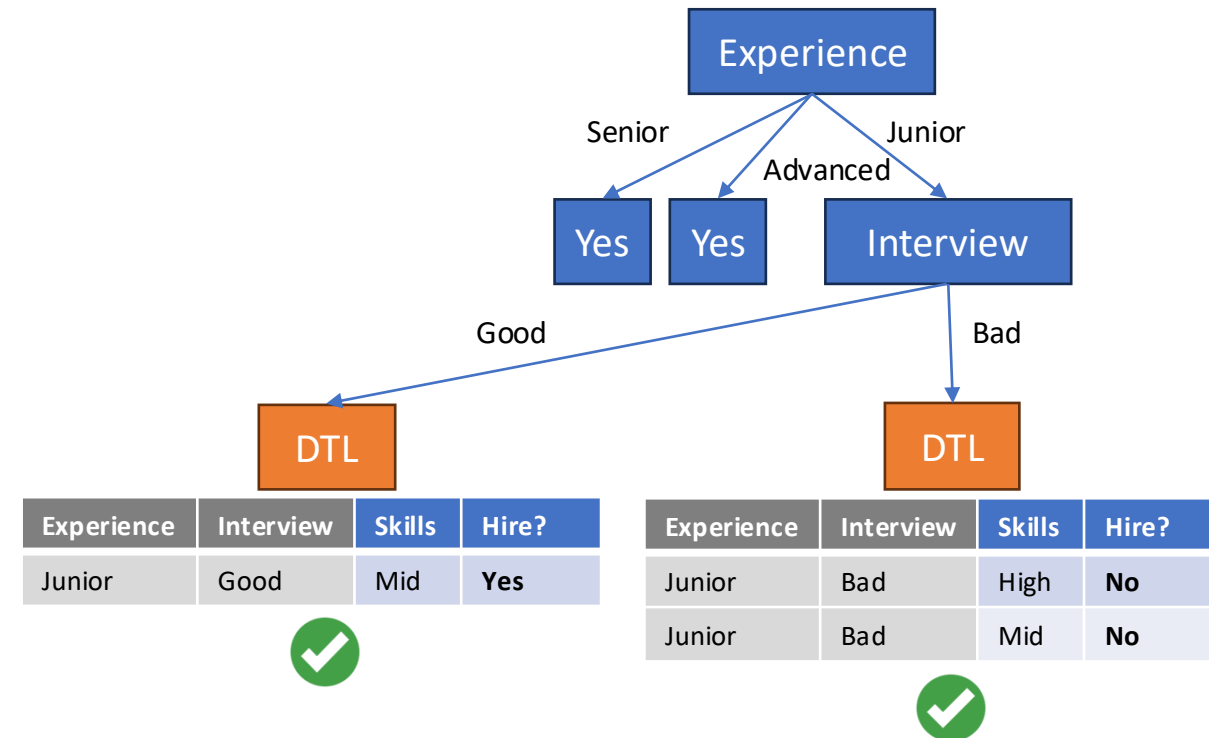
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):  
    if rows is empty: return default  
    if rows have the same classification:  
        return classification  
    if attributes is empty:  
        return majority_decision(rows)  
  
    best_attribute = choose_attribute(attributes, rows)  
    tree = a new decision tree with root best_attribute  
    for valuei of best_attribute:  
        subrowsi = {rows with best_attribute = valuei}  
        subtreei = DTL(subrowsi,  
                        attributes - best_attribute,  
                        majority_decision(rows))  
        add subtreei as a branch to tree with label valuei  
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

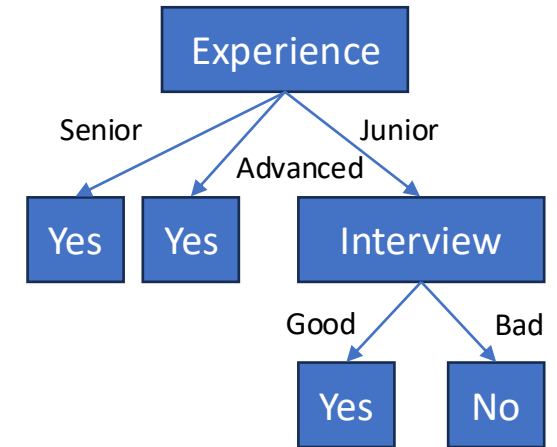
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Poll Everywhere

PollEV.com/**rizki**

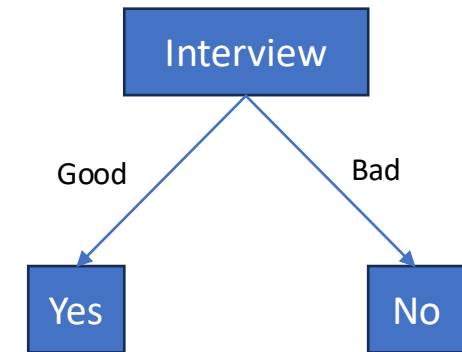
Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Junior	Bad	Low	No
Senior	Good	High	Yes
Advanced	Bad	Mid	No
Junior	Good	Mid	Yes
Advanced	Bad	High	No

Suppose you learn a decision tree model using decision tree learning algorithm and the data provided.

What is the root node of the decision tree model?

Poll Everywhere

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Junior	Bad	Low	No
Senior	Good	High	Yes
Advanced	Bad	Mid	No
Junior	Good	Mid	Yes
Advanced	Bad	High	No



Suppose you learn a decision tree model using decision tree learning algorithm and the data provided.

What is the root node of the decision tree model?

Decision Tree – Analysis

Theorem (Representational Completeness of Decision Tree):

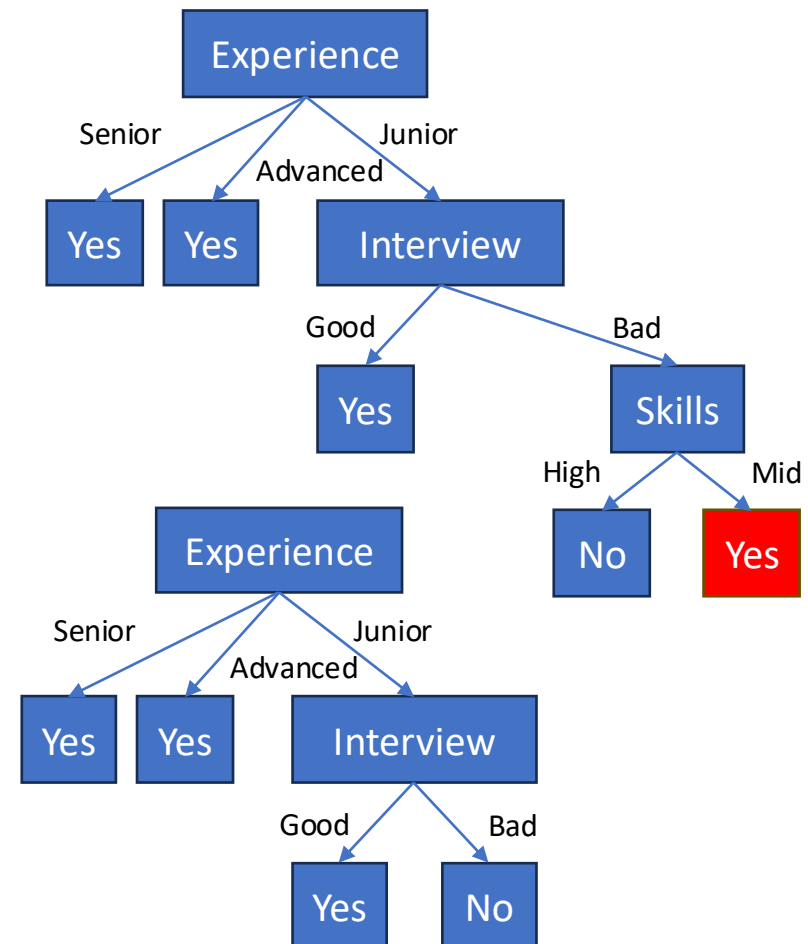
For any finite set of **consistent** examples with discrete (categorical) features and finitely many label classes, there exists a **decision tree that perfectly classifies them**.

TL;DR (imprecise): a decision tree can fit “any” training data.

- **Benefit:** **strong!** can capture genuine patterns, even complicated ones.
- **Drawback:** may also capture any **noise or outliers**.

Fitting the noise makes the tree needlessly complex, a problem known as **overfitting**.

No	Experience	Interview	Skills	Hire?
1	Senior	Good	High	Yes
2	Advanced	Bad	High	Yes
3	Junior	Good	Mid	Yes
4	Junior	Bad	High	No
5	Junior	Bad	Mid	Yes



Pruning

Pruning is the process of **removing parts of a tree** to produce a simpler (more robust) tree that generalizes better to unseen data by limiting its representational capability. It makes the tree less **stronk!**

Common pre-pruning (early stopping) methods:

- **Max depth:** limits how deep the tree can grow. A tree's depth is the length of the longest path from the root to a leaf.
Motivation: Deeper trees make finer, more specific splits that tend to memorize individual examples.
- **Min-sample leaves:** sets a minimum number of samples a leaf must contain. Leaves with fewer samples than this threshold are pruned.
Motivation: A leaf with very few samples makes decisions based on little evidence, making it unreliable and sensitive to noise.

Pruning can also be done after the tree is fully grown—called **post-pruning**—which is beyond the scope of this course.

Pruning: Max-Depth

Max-depth = 1

Depth 0

DTL

Depth 1

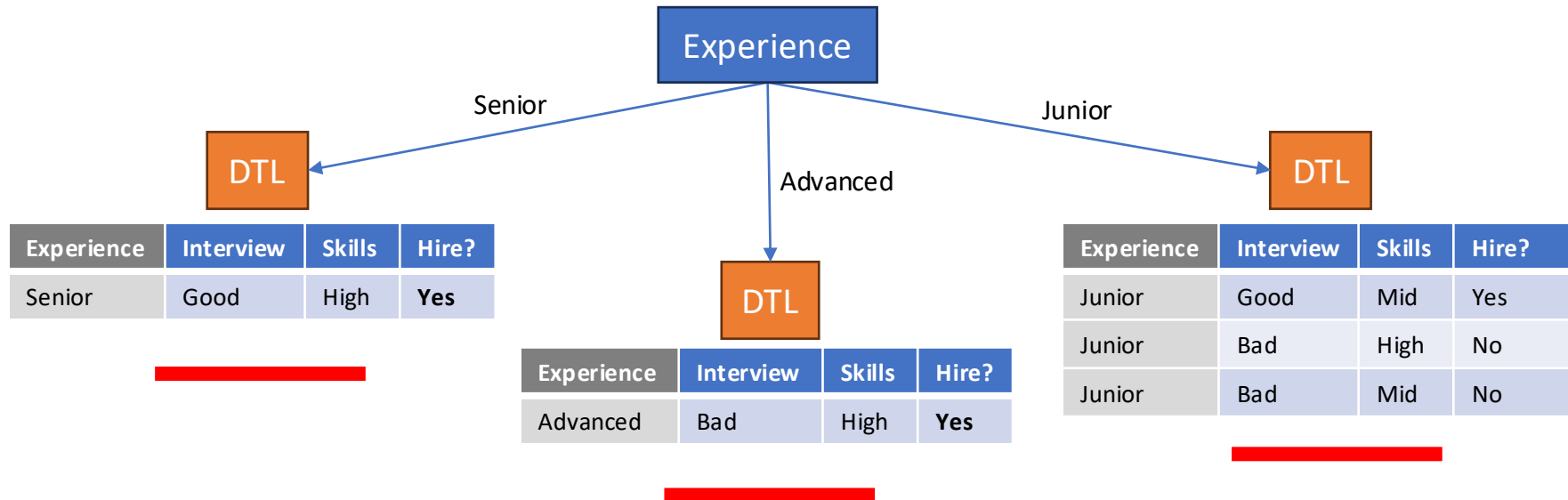
Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Max-Depth

Max-depth = 1

Depth 0

Depth 1



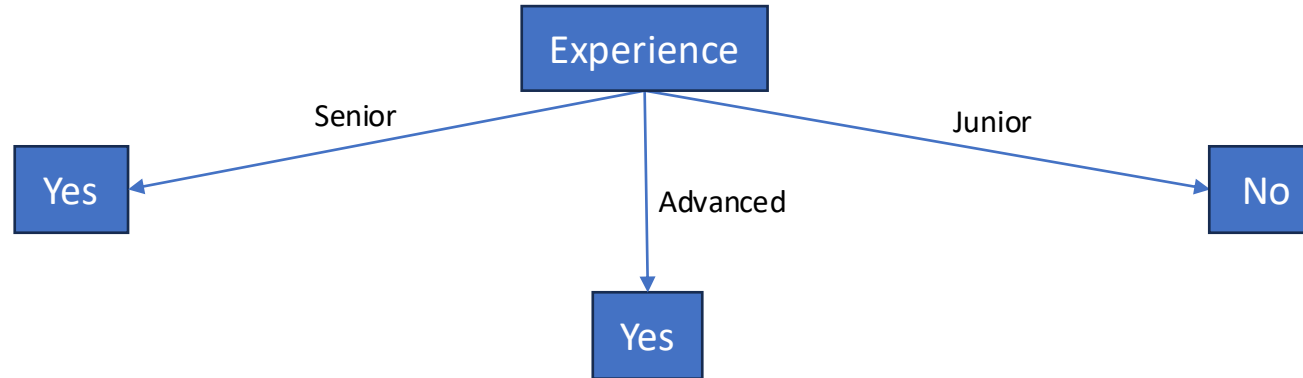
Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Max-Depth

Max-depth = 1

Depth 0

Depth 1



Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Min-Sample Leaves

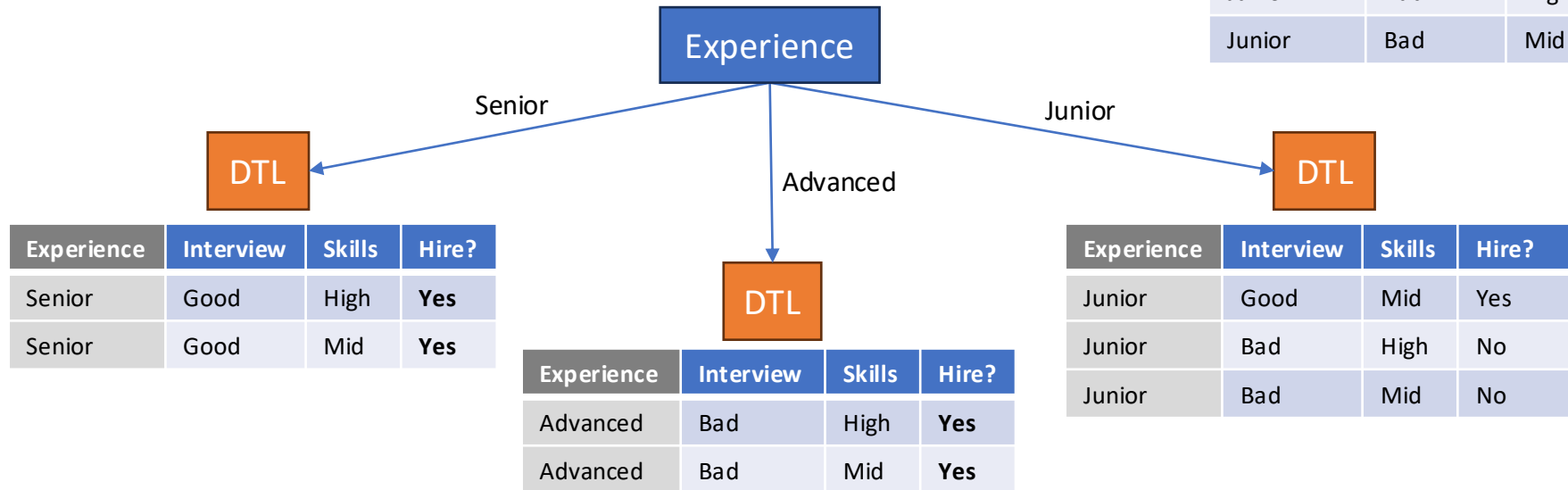
Min-sample leaves = 2

DTL

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Senior	Good	Mid	Yes
Advanced	Good	High	Yes
Advanced	Good	Mid	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Min-Sample Leaves

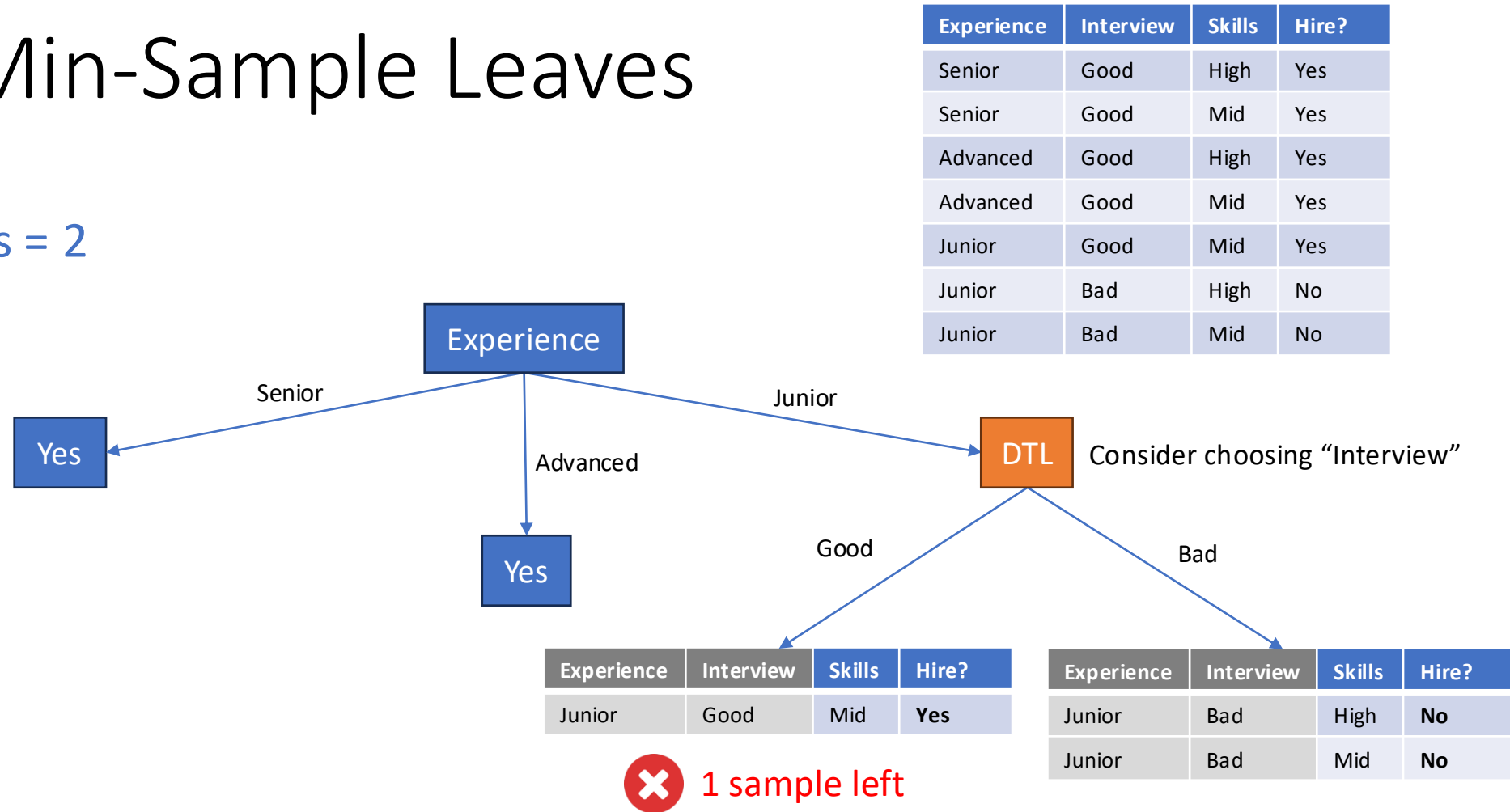
Min-sample leaves = 2



Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Senior	Good	Mid	Yes
Advanced	Good	High	Yes
Advanced	Good	Mid	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Min-Sample Leaves

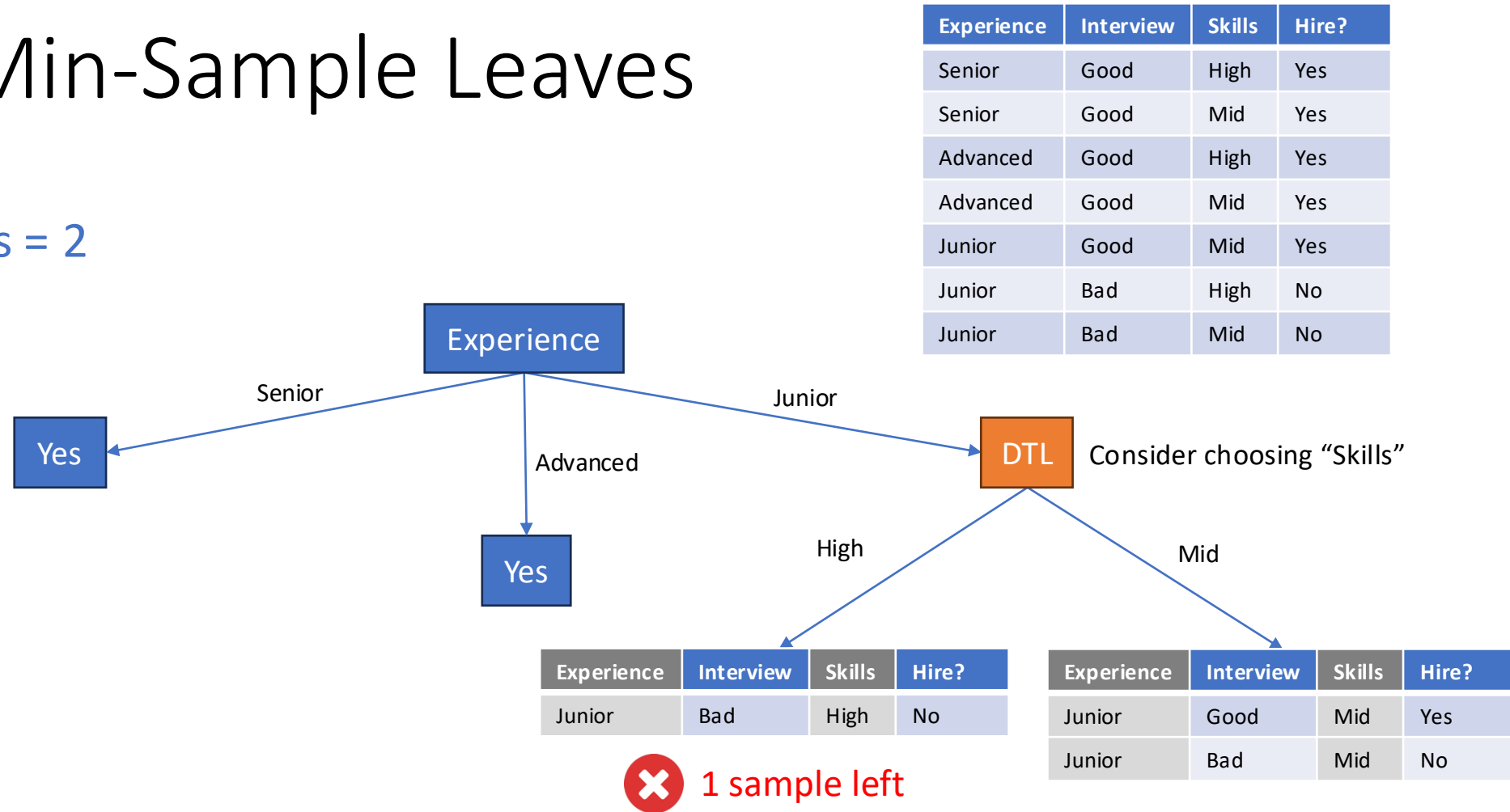
Min-sample leaves = 2



Splitting is not possible,
Need to consider the next best attribute.

Pruning: Min-Sample Leaves

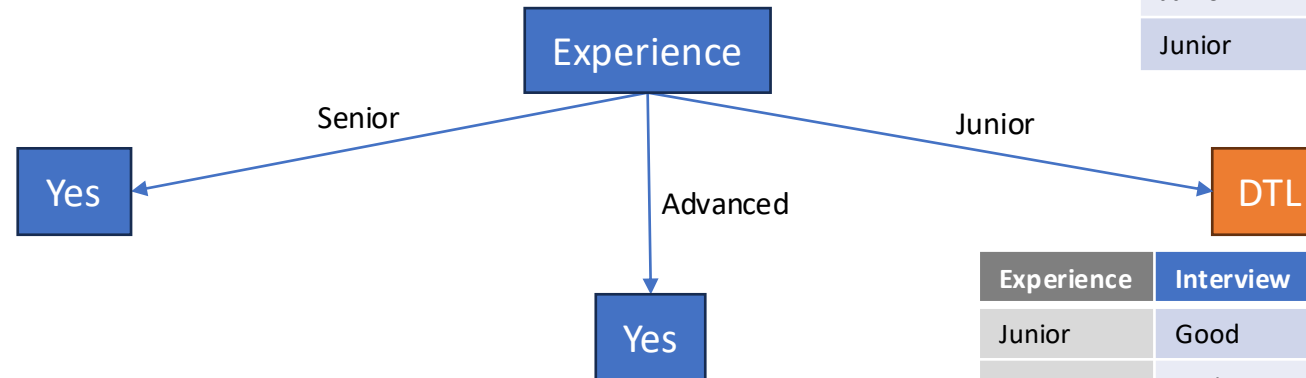
Min-sample leaves = 2



Splitting **still** not possible and no other attribute left.

Pruning: Min-Sample Leaves

Min-sample leaves = 2



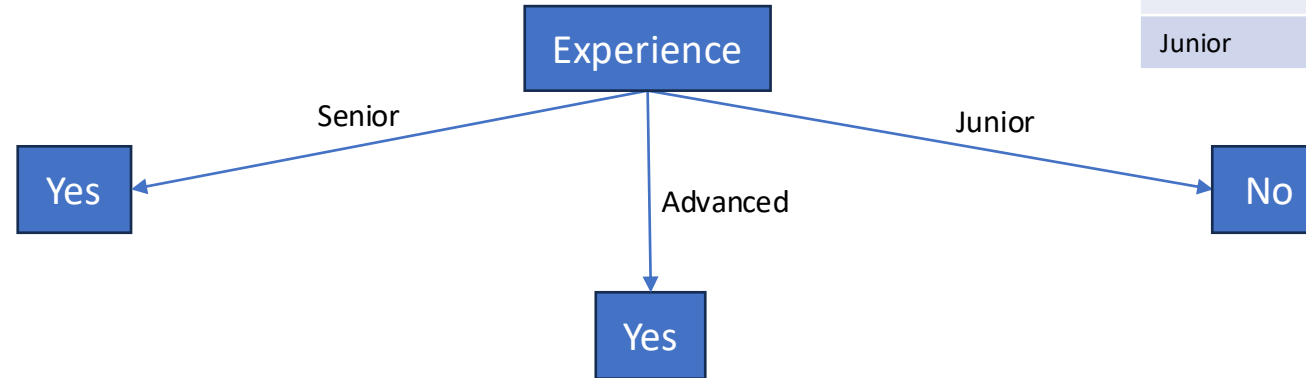
Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Senior	Good	Mid	Yes
Advanced	Good	High	Yes
Advanced	Good	Mid	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Experience	Interview	Skills	Hire?
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Choose **majority** or **default**.

Pruning: Min-Sample Leaves

Min-sample leaves = 2



Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Senior	Good	Mid	Yes
Advanced	Good	High	Yes
Advanced	Good	Mid	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Real-World Data

Real-world data is often messy and **may contain attributes or values that are incompatible** with decision tree models, such as attributes with continuous values or missing values. In such cases, [preprocessing](#) the data is essential.

- **For attributes with continuous values:**

- Partition the values into a discrete set of intervals.
- Example:
Experience (viewed as a real-valued time period) can be partitioned into Junior (<1 year), Advanced (1-3 years), Senior (> 3 years).

- **For attributes with missing values:**

- Assign the most common value of the attribute
- Assign the most common value of the attribute with the same output
- Assign probability to each possible value and sample
- Drop the attribute
- Drop the rows
- ...

Decision Trees in Real World

- **XGBoost/LightGBM/CatBoost: combine many decision trees (ensemble).** Powering a very large share of winning solutions on Kaggle, especially for structured/tabular data problems.
- **PayPal, American Express, and other banks** use decision trees for credit scoring and fraud detection.
- Heavily used in **bioinformatics and genomics** for tasks like gene selection and disease classification.

Resources

Scikit Learn

- Decision Trees
 - <https://scikit-learn.org/stable/modules/tree.html>
- Ensembles
 - <https://scikit-learn.org/stable/modules/ensemble.html>

Summary

- Machine Learning
 - Designing an agent for difficult problems or problems with difficult to specify rules.
 - Machine learning: ability to learn and improve from data/experience.
 - Types: supervised, unsupervised, semi-supervised, reinforcement
 - Supervised Learning: learning with labels
 - Data, Model, Loss
 - Regression, Classification
 - Performance measures:
 - Classification: Accuracy, Precision, Recall, F1 score.
 - Regression: MSE, MAE.
- Decision Trees
 - Decision Tree Model
 - Entropy, conditional entropy, information gain
 - Decision tree learning algorithm (DTL): Greedy maximization of the information gain.
 - Pruning: max depth, min-sample leaves
 - Real-World Data is messy, need preprocessing

Further Reading (Optional)

- Representation Completeness of Decision Tree Model
 - Mitchell, T. M. (1997). *Machine Learning*, Chapter 3. - Quinlan, J. R. (1986). "Induction of decision trees." *Machine Learning*, 1(1), 81–106.
- Optimal Decision Tree
 - Hyafil, L., & Rivest, R. L. (1976). "Constructing optimal binary decision trees is NP-complete." *Information Processing Letters*, 5(1), 15–17. - Mitchell, T. M. (1997). *Machine Learning*, Chapter 3.
- Post-pruning methods (TESL, Chapter 9)
- Gradient boosted tree (TESL, Chapter 10)
- Random forests (TESL, Chapter 15)

Coming Up Next Week

- Linear Regression
- Learning Algorithms:
 - Gradient Descent
 - Normal Equation

To Do

- **Lecture Training 4**
 - +550 EXP
 - +100 Early bird bonus
- **Problem Set 2**
 - Will be released (hopefully) later today!